BAB I
CLASS DAN OBJEK

A. TUJUAN

1. Mahasiswa mampu mengetahui dan memahami konsep class dan objek
2. Mahasiswa mampu memahami penggunaan method dalam class
3. Mahasiswa mampu memahami attribute class.
4. Mahasiswa mampu mengimplementasikan class dan objekk berdasarkan kasus yang diberikan menggunakan bahasa pemrograman Java.

B. ALAT DAN BAHAN

1. Laptop
2. Mouse
3. IDE Java (Netbeans 14)

C. KESEHATAN DAN KESELAMATAN KERJA

1. Hati-hatilah dalam memakai perangkat elektronik
2. Pastikan kabel listrik terpasang dan dalam kondisi baik
3. Lakukan praktikum dalam posisi duduk yang benar
4. Jauhkan kabel listrik dari sentuhan anda
5. Gunakan alas kaki celana Panjang dan kemeja
6. Gunakan kacamata anti radiasi layar

D. TEORI

1. Class dan object pada Java

Berbicara mengenai OOP maka kita tidak akan lepas dari konsep class dan object karena class dan objet adalah aspek utama dalam OOP. Perhatikan ilustrasi berikut:

Class	Object
Buah	Pisang Mangga Rambutan

Contoh lain

Class	Object
Laptop	Asus HP Lenovo

Berdasarkan ilustrasi di atas dapat disimpulkan bahwa class adalah "template" dari sebuah object dan sebuah object adalah hasil cetakan dari class. Ketika sebuah object dibuat maka object tersebut mewarisi semua variable dan method yang ada pada class pembentuknya.

Sintaks 2.1. membuat object dari class

```
nama_class nama_object = new nama_class();
```

Keterangan sintaks:

- nama_class: pada bagian ini diisi dengan nama class yang akan anda buat objectnya.
- nama_object: pada bagian ini diisi dengan nama object dari class yang diinginkan. Nama class bebas asal sesuai dengan aturan pemberian nama.
- new: merupakan keyword pada sintaks pendeklarasian object
- nama_class(): merupakan nama class yang sama dengan nama_class namun diikuti dengan ().

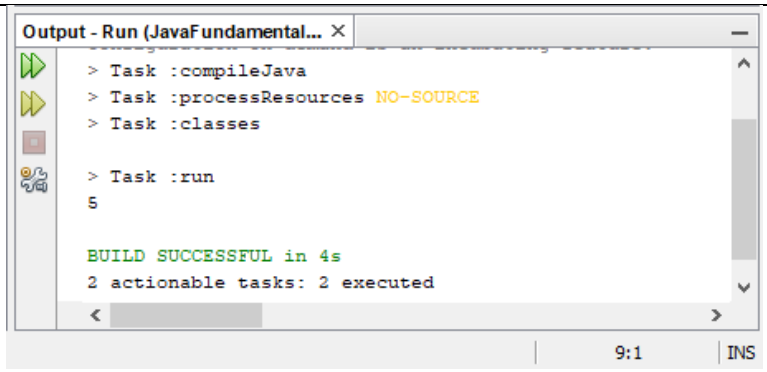
Contoh 2.1. Class dengan satu object

1. Modifikasi project JavaFundamentalI.
2. Pada class Main hapus kode sebelumnya yang sudah ada lalu ketikkan kode berikut:

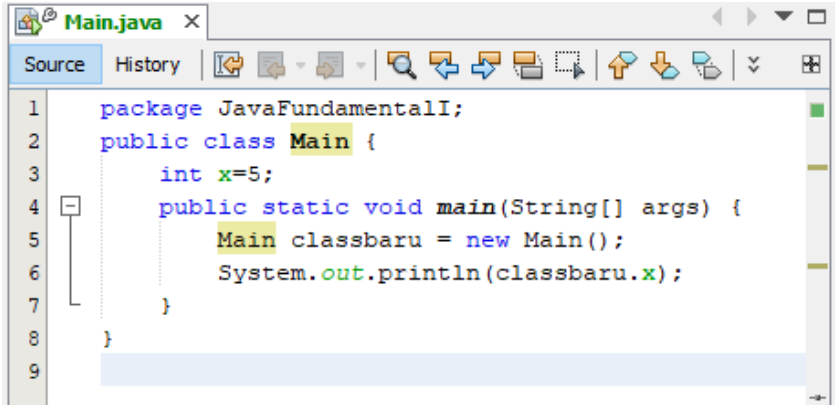
```
package JavaFundamentalI;


public class Main {
    int x = 5;
    public static void main(String[] args) {
     Main classbaru = new Main();
      System.out.println(classbaru.x);
    }
}
```

3. Kemudian jalankan atau tekan F6.
4. Maka akan muncul output sebagai berikut:



Berikut adalah gambar source code lengkap setelah anda melakukan langkah ke-2 dan ke-3 pada contoh 2.1.



Gambar 2.1. Screenshot Main.java contoh 2.1.

Pada gambar dapat dilihat bahwa pada **baris 3** kita telah mendeklarasikan sebuah variable bernama "x" dengan tipe data integer. Variable tersebut diisi dengan nilai 5. Jika diamati, perintah pendeklarasian variable ini dilakukan di luar area method main, namun masih berada di dalam class Main. Kemudian pada **baris 5,** terdapat statement membuat sebuah object dari class Main. Object tersebut bernama "classbaru" yang merupakan cetakan dari class Main, sehingga object classbaru akan mewarisi semua method dan variable member dari class Main.

Pada **baris 6**, diberikan sebuah baris perintah untuk mencetak "classbaru.x". Statement "classbaru.x" berarti bahwa variable x yang berada di dalam object classbaru. Object classbaru memiliki variable member x karena mewarisi class Main.

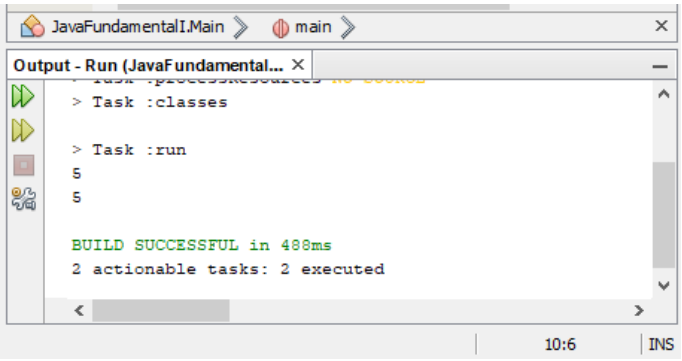
Contoh 2.2. Class dengan lebih dari satu object

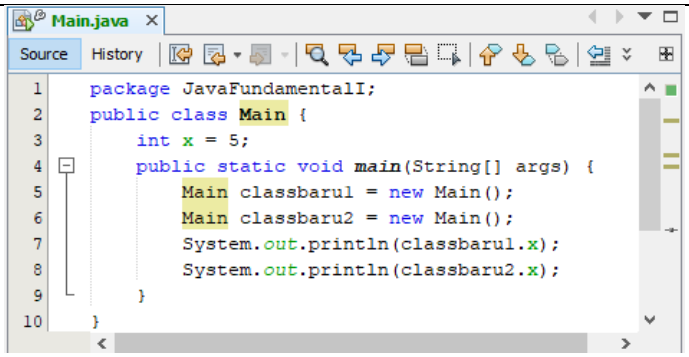
1. Modifikasi project JavaFundamentalI.
2. Pada method main di dalam class Main tambahkan kode berikut:

```
package JavaFundamentalI;
```

```
public class Main {
    int x = 5;
    public static void main(String[] args) {
        Main classbaru1 = new Main();
        Main classbaru2 = new Main();
        System.out.println(classbaru1.x);
        System.out.println(classbaru2.x);
    }
}
```

3. Kemudian jalankan atau tekan F6.
4. Maka akan muncul output sebagai berikut:



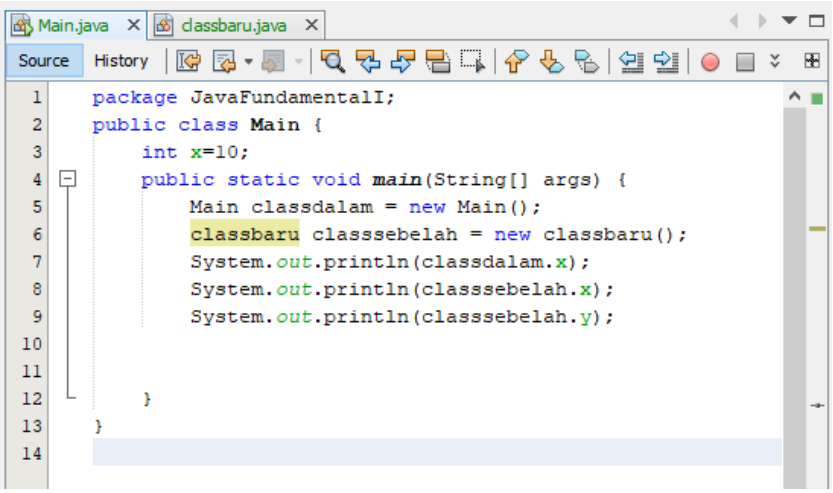


Gambar 2.2. Screenshot Main.java contoh 2.2.

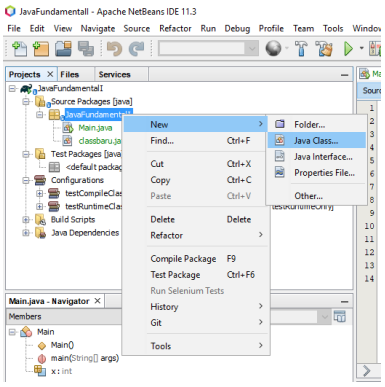
Pada gambar 2.2 dpat kita lihat source code lengkap dari contoh 2.2. pada contoh ini anda telah membuat 2 object dari sebuah class. Pada **baris 5** kita mendeklarasikan object dengan nama "classbaru1" dari class Main. Pada **baris 6** kita mendeklarasikan object dengan nama "classbaru2" dari class Main juga. Dapat kita lihat pada bagian output, hasil yang dikeluarkan adalah sama yaitu 5. Hal tersebut terjadi karena kedua object dibentuk dari class yang sama dengan mengakses variabel member yang sama menggunakan perintah **baris 7 dan 8**.

Contoh 2.3. Multiple class dan multiple object

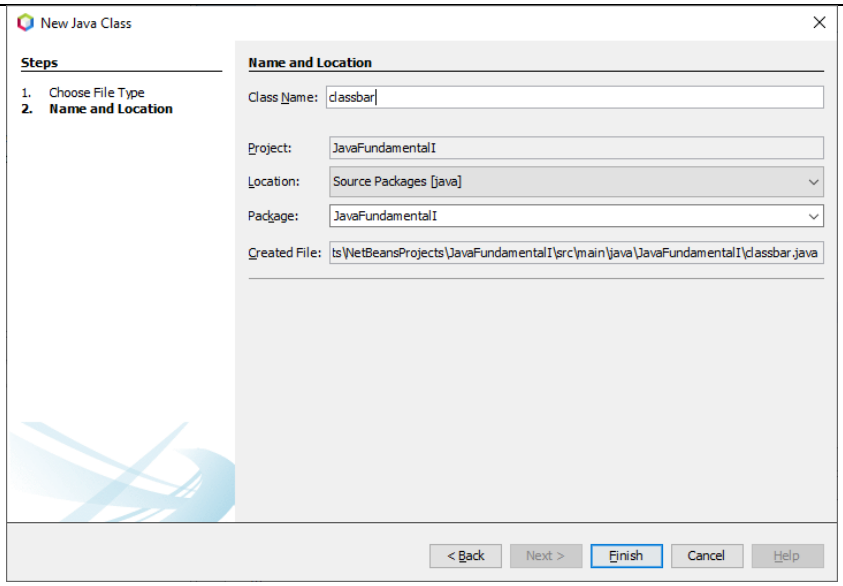
1. Modifikasi project JavaFundamentalI.
2. Pada method main di dalam class Main tambahkan kode berikut:



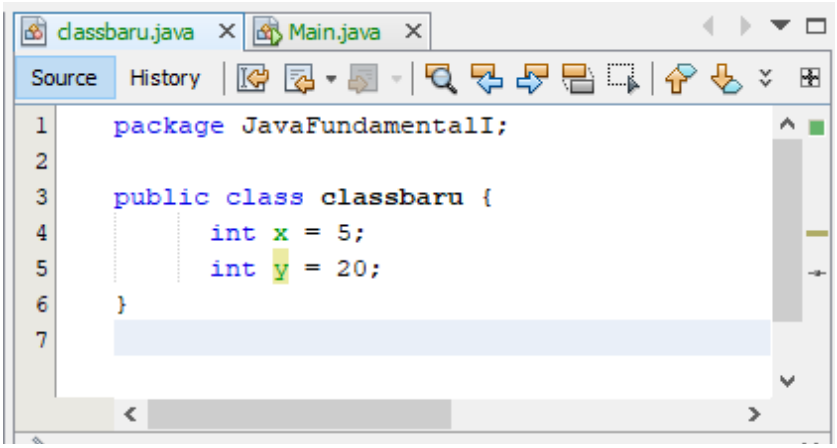
3. Kemudian tambahkan file class baru dengan cara klik kanan package class>new>Java Class



4. Beri nama class baru dengan nama "classbaru"

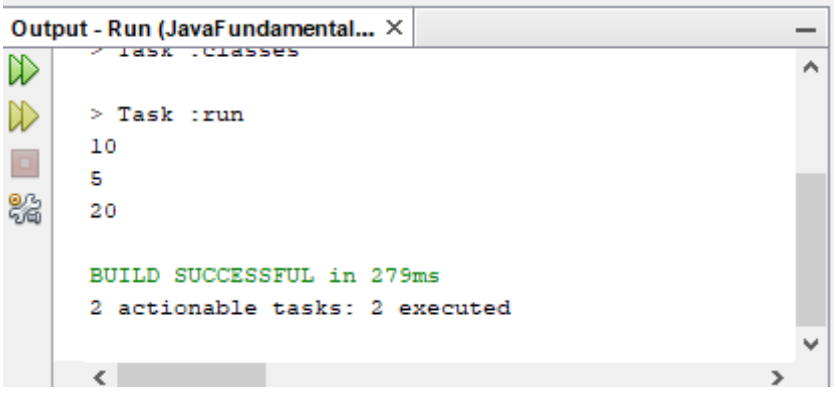


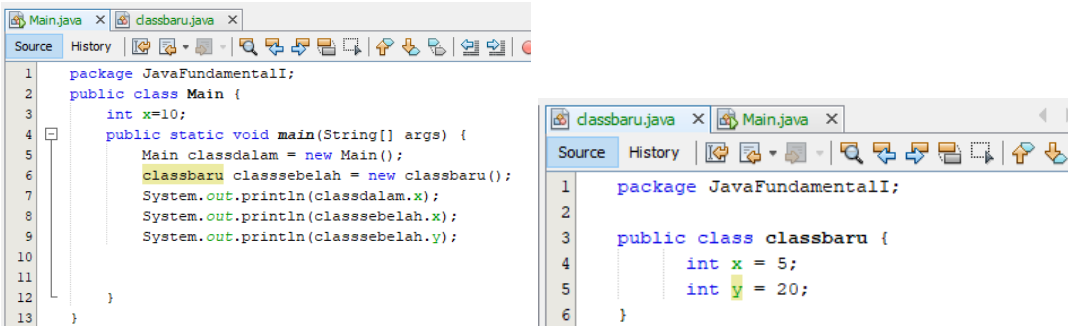
5. Pada classbaru.java tambahkan kode berikut:



6. Kemudian jalankan atau tekan F6.

7. Maka akan muncul output sebagai berikut:





Gambar 2.3. Screenshot Main.java dan classbaru.java contoh 2.3.

Pada contoh 2.3 di atas, kita telah menambahkan sebuah file class baru bernama "classbaru.java". Sehingga dalam package JavaFundamentalI terdapat 2 file class yaitu Main.java dan classbaru.java. pada class Main terdapat method main() sehingga ketika program dieksekusi, yang pertama kali akan dieksekusi adalah pada bagian tersebut. Pada class Main di **baris 3** dideklarasikan sebuah variable "x" bertipe integer dengan nilai 10. Pada **baris 5** dibentuk sebuah object bernama "classdalam" dimana object ini mewarisi class Main. Sehingga ketika nilai variable x pada object classdalam ditampilkan pada **baris 7**, nilai yang ditampilkan adalah 10.

Adapun object "classsebelah" yang dibentuk pada **baris 6** adalah mewarisi class classbaru yang berada pada file classbaru.java. classbaru memiliki 2 variabel yang dideklarasikan pada **baris 4 dan 5** pada classbaru.java, yakni x dan y yang bertipe integer dan masing masing menampung nilai 5 dan 20. Ketika object classsebelah dibentuk dari class classbaru, maka secara otomatis object classsebelah mewarisi semua "sifat" dari classbaru. Sehingga, ketika nilai x dan y dari object classsebelah ditampilkan menggunakan baris perintah **baris 8 dan 9**, nilai yang ditampilkan sama dengan nilai yang ada pada classbaru.

2. Class method dan atribut

Sebuah method adalah sebuah blok kode yang hanya akan berjalan atau dieksekusi ketika method ini dipanggil. Anda dapat mengirim data yang disebut parameter kedalama sebuah method. Method digunakan untuk melakukan aksi tertentu yang dikenal dengan sebutan fungsi. Method penting untuk digunakan agar kita dapat menggunakan kembali kode yang telah dibuat. Kita cukup membuat sebuah method dan kita dapat menggunakannya berulang kali.

Sebuah method harus dideklarasikan di dalam sebuah class. Method didefenisikan dengan nama method diikuti dengan buka dan tutup kurung "()". Pada Java terdapat method bawaan seperti "System.out.println()" namun anda bisa mendefenisikan sendiri method anda. Adapun sintaks pendeklarasian method secara umum adalah sebagai berikut:

Sintaks 2.2. Deklarasi method

```
Modifier tipe_nilai_balik nama method(){
  // blok kode yang akan dieksekusi
}
```

Contoh deklarasi method

```
public class MyClass {
  static void myMethod() {
    // blok kode yang akan dieksekusi
  }
}
```

Berdasarkan sintaks dan contoh yang telah diberikan dapat diketahui bahwa sebuah method terdiri atas:

- Modifier (static / non static): method dengan keyword static berarti pertanda bahwa method tersebut berada di dalam class tersebut namun bukan merupakan turunan dari class tersebut. Method ini dapat dipanggil tanpa turunan atau object dari class tersebut. Sedangkan metode non static dalam pemanggilannya harus melalui object class yang sudah dibentuk terlebih dahulu. Untuk memanggil non static method, kita perlu menuliskan nama dari method dan diikuti dengan nama object class.
- Tipe_nilai_balik: merupakan jenis nilai balik yang dikembalikan. Jika tidak ada nilai balik dari method tersebut maka cukup memberikan void.
- Nama_method: merupakan nama dari method yang dibangun. nama yang diberikan sebaiknya merupakan gambaran dari isi method.

Contoh 2.4. Static method

1. Modifikasi project JavaFundamentalI.
2. Pada class Main.java tambahkan kode berikut:

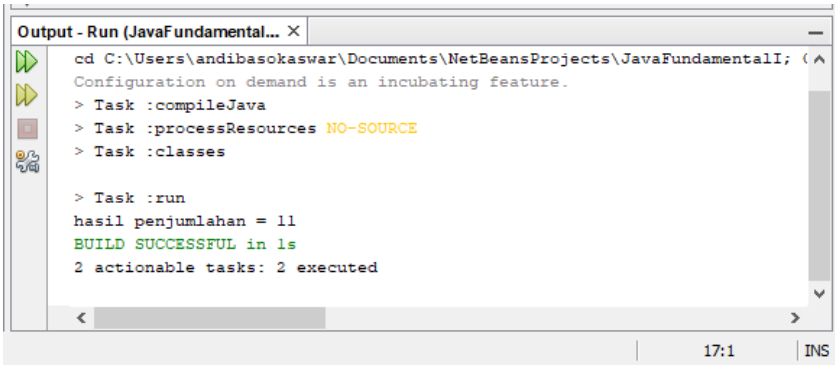
```
package JavaFundamentalI;

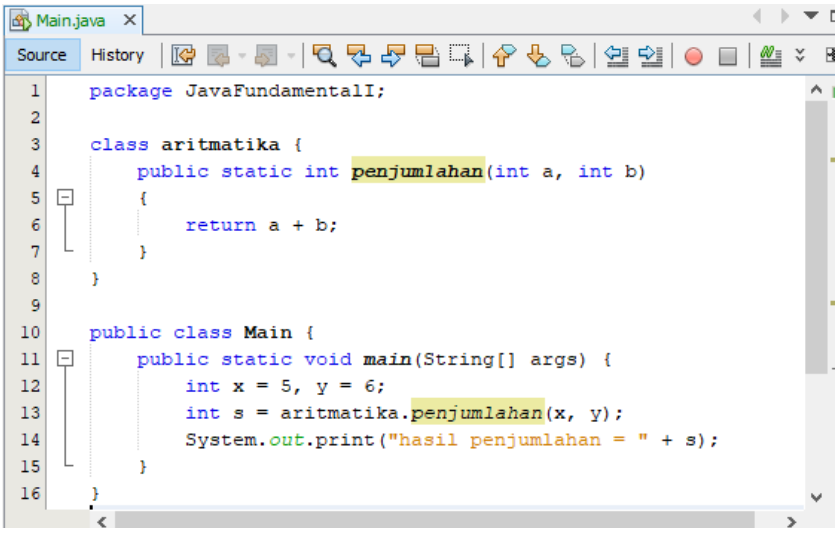
class aritmatika {
```

```
    public static int penjumlahan(int a, int b)
    {
        return a + b;
    }
}

public class Main {
    public static void main(String[] args) {
        int x = 5, y = 6;
        int s = aritmatika.penjumlahan(x, y);
        System.out.print("hasil penjumlahan = " + s);
    }
}
```

3. Kemudian jalankan atau tekan F6.
4. Maka akan muncul output sebagai berikut:





Gambar 2.4. Screenshot Main.java dan classbaru.java contoh 2.4.

Pada contoh 2.4 anda telah membuat sebuah program penjumlahan dengan menggunakan static method. Pada program tersebut terdapat dua class, yaknik class aritmatika dan class Main. Method utama berada pada class Main. Untuk class aritmatika sendiri area blok kodenya dimulai dari **baris 3 hingga 8**. Di dalam class aritmatika, pada **baris 4 hingga 7**, terdapat sebuah method static bernama "penjumlahan" yang bersifata public (dapat diakses dari manapun baik dalam maupun luar class) dengan tipe nilai balik adalah integer. Pada method ini terdapat dua parameter yaitu "a" dan "b" yang masing masing bertipe data integer. Kemudian didalam class aritmatika method penjumlahan() ini terjadi proses penjumlahan (**baris 6**) antara isi variable a dan b yang langsung dikembalikan (return) ke perintah pemanggilnya di class Main **baris 13**.

Pada class Main.dengan areanya dari baris **10 hingga 16** terdiri atas sebuah method bernama main bersifat public dan static namun tidak memiliki nilai balik (void). Pada method ini dilakukan deklarasi variable pada **baris 12** yaitu "x" dan "y" yang sama sama bertipe integer dimana x bernilai 5 dan y bernilai 6. Kemudian pada **baris 13** dilakukan deklarasi variable "s" yang bertipe data integer.

Nilai variable s diperoleh dari nilai balik hasil pemanggilan method penjumlahan() yang berada dalam class aritmatika. Pemanggilan method ini disertai pengiriman nilai parameter yaitu x dan y yang kemudian nilai x dan y akan diterima oleh variable a dan b secara berurut pada method penjumlahan() yang berada di class aritmatika. Variable s dideklarasikan dengan tipe data integer dikarenakan nilai balik yang diterima hasilnya berupa integer juga. Isi variable s kemudian ditampilkan menggunakan perintah pada **baris 14**. Jika anda **MENGHAPUS** keyword static pada method penjumlahan di class aritmatika, maka compiler akan menampilkan pesan error.

Pada contoh di atas kita telah dapat mengetahui bahwa method static dapat diakses tanpa perlu membentuk objectnya terlebih dahulu.

Selanjutnya adalah class attribute. Class attribute pada dasarnya adalah sebuah variable yang berada di dalam sebuah class.

Sintaks 2.3. Deklarasi method

```
Modifier class nama class(){
  //variable yang dideklarasikan
}
```

Contoh deklarasi class attribute

```
public class Main {
   int x = 5;
   int y = 3;
}
```

E. PRAKTIKUM

Praktikum 2.1.

```
package com.mycompany.praktikum21;
class Lampu {
    boolean isOn;
  void turnOn() {
    isOn = true;
    System.out.println("Lampu menyala? " + isOn);
  }

  void turnOff() {
    isOn = false;
    System.out.println("Lampu menyala? " + isOn);
  }
}

public class Praktikum21 {
```

```
    public static void main(String[] args) {

    Lampu led = new Lampu();
    Lampu halogen = new Lampu();

    led.turnOn();
    halogen.turnOff();
    }
}
```

Praktikum 2.2.

```
package com.mycompany.praktikum22;

import java.util.Scanner;

public class Praktikum22 {
    final double PHI = 3.14d;
    double jariJari;
    double tinggi;

    public static void main(String[] args) {
        Praktikum22 kerucut = new Praktikum22();
        Scanner bacaInput = new Scanner(System.in);

        System.out.print("Masukkan nilai jari-jari alas kerucut:
");
        kerucut.jariJari = bacaInput.nextDouble();

        System.out.print("Masukkan nilai tinggi kerucut: ");
        kerucut.tinggi = bacaInput.nextDouble();

        double nilaiVolume = kerucut.volumeKerucut(kerucut.PHI,
kerucut.jariJari, kerucut.tinggi);
        System.out.print("Volume kerucut: "+nilaiVolume);
```

```
    }

    public double alasKerucut(double phi, double r){
        double luasAlas = phi * Math.pow(r,2);
        return luasAlas;
    }

    public double volumeKerucut(double phi, double r, double
tinggi){
        double volume = (alasKerucut(phi, r)*tinggi)/3.0;
        return volume;
    }
}
```

Praktikum 2.3.

```
package com.mycompany.praktikum23;

class Bangundatar{
    public void luasPersegi(float panjangSisi){
        float luas = panjangSisi * panjangSisi;
        System.out.println("Luas persegi adalah: " +luas);
    }

    public void luasPersegiPanjang(float panjang, float lebar){
        float luas = panjang * lebar;
        System.out.println("Luas persegi panjang adalah: "
+luas);
    }
};

class Bangunruang{
    public void volumeKubus(float panjangSisi){
        float volume = panjangSisi * panjangSisi * panjangSisi;
        System.out.println("Volume kubus adalah: " +volume);
```

```
    }

    public void volumeBalok(float panjang, float lebar, float
tinggi){
        float volume = panjang * lebar * tinggi;
        System.out.println("Volume balok adalah: " +volume);
    }

};

public class Praktikum23 {
    public static void main(String[] args) {
        Bangunruang kubus = new Bangunruang();
        kubus.volumeKubus(5.0f);

        Bangundatar persegipanjang = new Bangundatar();
        persegipanjang.luasPersegiPanjang(5.0f, 2.0f);
    }
}
```

F. SOAL TANTANGAN

1. Perhatikan bangun datar berikut ini. Buatlah sebuah program untuk mengetahui berapa luas area yang berwarna biru. Informasi terkait ukuran/dimensi pembantu dapat dilihat pada gambar. Program yang dibuat wajib menerapkan konsep class, object dan method.

